Tema 8. Formularios

1. Principios generales del diseño de formularios

Un formulario es una interfaz que hace de puente entre el usuario y el sistema al recoger este la información traspasada por el propio usuario.

Para ello los principios generales para un buen formulario buscan facilitar la interacción, reducir la tasa de abandono y asegurar la precisión de datos.

Para montar un formulario de cero usamos la etiqueta `<form> </form>`, en su interior usaremos `<div></div>` para hacer tantos bloques como queramos, donde pondremos en su interior un `<label></label>` y un `<input></input>` para crear el campo con una etiqueta acompañándolo.

2. Organización de los campos

Empezamos por decidir que campos se deben incluir en el formulario. Cada pregunta tiene un coste: para la organización, que debe programarla, procesar y almacenar el dato, y para el usuario, que invierte tiempo y esfuerzo en responderla. La guía de diseño del Gobierno de Escocia propone un *protocolo de preguntas*: solo se incluye un campo si sabemos para qué necesitamos el dato, qué haremos con él y cómo lo mantendremos actualizado y seguro.

Una vez claro el *qué*, llega el *cómo*, y no es solo una cuestión estética: un estudio de Seckler y su equipo (CHI 2014), citado por Nielsen Norman Group, mostró que el 78 % de los usuarios enviaba sin errores al primer intento los formularios que seguían las pautas de usabilidad, frente al 42 % en los que no las seguían.

Una sola columna. Con varias columnas, la vista salta en zigzag y es fácil saltarse campos; una sola columna crea un recorrido claro y se adapta sin cambios al móvil. Solo comparten fila los datos cortos que se perciben como una unidad, como ciudad y código postal.

Agrupar lo relacionado. Cada etiqueta va pegada a su campo, y el espacio entre grupos es mayor que dentro de ellos. En HTML, los grupos se marcan con `<fieldset>` y `<legend>`, y los lectores de pantalla anuncian el nombre del grupo:

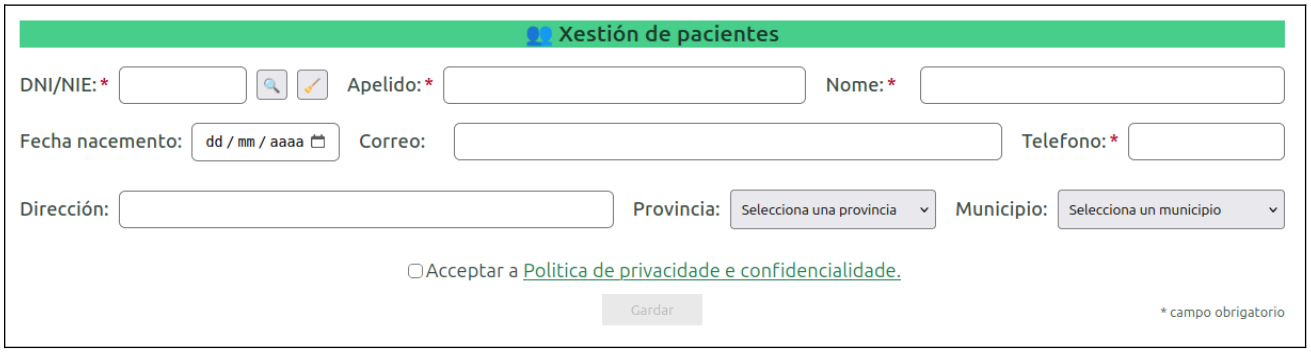
```
<fieldset>
  <legend>Dirección</legend>
  <label for="provincia">Provincia</label>
  <select id="provincia">…</select>
</fieldset>
```

Orden lógico. Los campos deben siguir el orden esperado como número de tarjeta → caducidad → código de seguridad. El orden visual debe coincidir con el del HTML, porque la tecla Tab y los lectores de pantalla siguen el código, no lo que se ve.

Mostrar solo lo necesario. Un campo que no aplica no debería verse: Por ejemplo Compañía aseguradora solo aparece si el paciente marca que tiene seguro privado.

Agrupar lo relacionado. Cada etiqueta va pegada a su campo, y el espacio entre grupos es mayor que dentro de ellos. En HTML, los grupos se marcan con `<fieldset>` y `<legend>`, y los lectores de pantalla anuncian el nombre del grupo:

Aplicación real: Xestión Pacientes. El formulario reparte los campos en tres columnas con etiquetas de distinto ancho, así que la vista zigzaguea y los campos no quedan alineados. Proponemos una sola columna con tres grupos (datos personales, contacto y dirección).



3. Etiquetas e instrucciones

Las etiquetas e instrucciones claras ayudan a los usuarios a entender lo que tienen que hacer. Las etiquetas deben ser breves y directas y ser lo suficientemente descriptivas pero poco genéricas para guiar mejor a los usuarios. Ejemplo de mal etiquetado donde no se menciona qué información hay que rellenar en cada campo:



4. Tipos de controles

Controles de selección

La diferencia entre botones de opción y casillas viene de las pautas de interfaz de Apple, *Inside Macintosh* (1984), y hoy forma parte de los estándares web. Respetar ese estándar permite al usuario predecir cómo funciona un control con solo verlo.

- **Botones de opción (radio):** para elegir una sola opción entre varias que se excluyen entre sí. Conviene preseleccionar una.
- **Casillas de verificación (checkbox):** se pueden marcar ninguna, una o varias, porque son independientes. Una casilla sola sirve para un sí o no.
- **Desplegables (select):** ocultan las opciones hasta que se abren, así que solo compensan cuando hay muchas.

La apariencia debe ser la estándar, las opciones van en vertical y la etiqueta también debe ser clicable. En HTML, los botones de opción con el mismo `name` forman un grupo, y envolver cada uno en su `<label>` hace clicable el texto:

```
<label><input type="radio" name="colexiado" value="si"> Sí</label>
<label><input type="radio" name="colexiado" value="non" checked> Non</label>
```

Controles de escritura

HTML5 ofrece tipos de campo que aportan, sin JavaScript, el teclado adecuado en el móvil y una validación básica. Un DNI, un teléfono o un código postal son códigos, no cantidades, así que nunca usan `number`. El tamaño del campo debe corresponder al dato esperado.Para textos largos se usa `<textarea>`.

Tipo	Qué aporta
`email`	Teclado con @ y comprobación básica del formato.
`tel`	Teclado telefónico; no valida el formato, que cambia según el país.
`date`	Calendario; el formato depende del idioma del navegador.
`number`	Flechas y límites `min` y `max`; solo para cantidades.

5. Campos obligatorios y opcionales

Un formulario puede tener campos obligatorios y opcionales. Es importante especificar los campos obligatorios con los atributos “required” para que el usuario no pueda entregrar el formulario sin antes haber rellenado el formulario. Por último hay que dejarle claro al usuario cuales son obligatorios y si se ha saltado uno.

6. Validación visual

La validación visual indica qué datos son correctos y cuáles no. Según Nielsen Norman Group, debe permitir que el usuario detecte el error, entienda cómo corregirlo y lo siga viendo mientras lo corrige.

Cuándo validar. Cuando el usuario termina de rellenar cada campo, no antes: marcar un correo como erróneo tras la primera letra confunde y molesta. En CSS, `:user-invalid` aplica el estilo de error solo después de que el usuario haya usado el campo. En campos complejos sí ayuda una señal continua, como la fortaleza de una contraseña mientras se escribe.

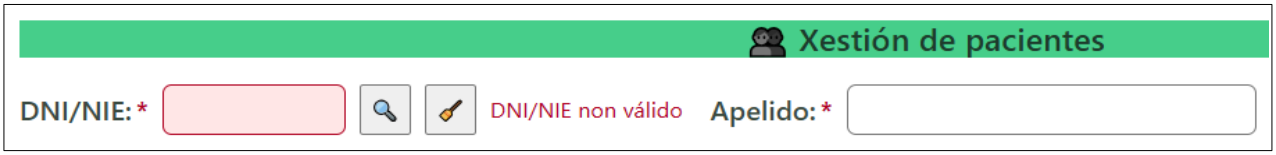
Dónde mostrar el error. Junto al campo, y visible mientras se corrige. Un resumen al principio del formulario es útil, pero nunca como única indicación. Si el error queda fuera de la pantalla al enviar, hay que llevar al usuario hasta él: un formulario largo que «no hace nada» al pulsar Enviar, con el error escondido arriba, solo genera frustración.

Nunca solo con color. Alrededor de 1 de cada 12 hombres tiene daltonismo rojo-verde y puede no notar que un campo pasa de rojo a verde. Por eso el color siempre va acompañado de un icono y un texto. Tampoco conviene mostrar los errores en ventanas modales, que hay que cerrar antes de poder corregir.

Para lectores de pantalla. `aria-invalid` marca el campo como erróneo y `aria-describedby` le asocia el mensaje, que se lee al llegar al campo:

```
<input type="email" id="correo" aria-invalid="true" aria-describedby="correo-error">
<p id="correo-error">⚠️ Falta la @. Ejemplo: nome@exemplo.gal</p>
```

Aplicación real: Xestión Pacientes. La aplicación muestra el error junto al campo, con color y con texto, así que no depende solo del color. Como mejoras: con tres columnas, el mensaje compite con el campo siguiente; el correo pordía aparece como non válido estando vacío, aunque no es obligatorio; y el botón Gardar aparece deshabilitado sin indicar qué falta.



7. Mensajes de error y confirmación

El mensaje de error más común a la hora de hacer un formulario salta justo al intentar confirmar la entrega. Aunque esta práctica cumpla con su propósito de informar al usuario de que el formulario no se ha entregado por errores en los campos es una medida que puede provocar frustración.

Por ello la manera ideal de avisar sobre errores es complementar la validación en tiempo real para corregir sobre la marcha los errores. De esta manera los usuarios tienen mensajes más específicos para entender qué tienen que rellenar en cada campo y reduciendo los errores. Por el otro lado, cuando el formulario se envía correctamente, es importante mostrar un mensaje de confirmación claro, indicando que los datos se han recibido y qué ocurrirá a continuación. Por ejemplo: Solicitud enviada correctamente. O Recibirás un correo de confirmación. También conviene evitar envíos duplicados y conservar los datos introducidos si se produce un error, para que el usuario no tenga que rellenarlos de nuevo.

8. Usabilidad de los formularios

La usabilidad es lo fácil y rápido que resulta rellenar un formulario. Además de lo visto en los apartados anteriores, hay tres factores clave.

Navegación con teclado. Muchos usuarios saltan de campo en campo con la tecla Tab. El recorrido debe seguir el orden visual y lógico, y el campo activo debe tener un foco visible.

Autorrelleno. El navegador puede rellenar datos que el usuario ya escribió en otras webs. Deduce qué espera cada campo a partir de su etiqueta y sus atributos, y el atributo `autocomplete` elimina las dudas:

```
<input type="email" id="correo" name="correo" autocomplete="email">
```

En las contraseñas no conviene desactivarlo: con `new-password`, el navegador propone una contraseña segura. El autorrelleno solo se desactiva cuando perjudica al usuario.

Sin botones de restablecer. Vaciar todo el formulario por error cuesta mucho más de lo que ahorra la rara necesidad de empezar de cero.

Aplicación real: Xestión Pacientes. Aquí el autorrelleno es la excepción que confirma la regla: el personal del centro introduce datos de otras personas, y el navegador ofrecería los del propio empleado o los de un paciente anterior en un ordenador compartido, así que `autocomplete="off"` está justificado.

9. Errores frecuentes

A continuación hay una lista de errores frecuentes a evitar para que no sean frustrantes:

- Demasiados campos: Una gran cantidad de campos a la vez puede abrumar al usuario. Para evitar esto sería más conveniente usar menos campos o separarlos en diferentes formularios que rellenar uno a la vez.
- Etiquetas poco claras: Los usuarios no sabrán qué tienen que rellenar en cada campo si el etiquetado es vago o nulo.
- Mala alineación: Campos desalineados sin una estructura clara dará la impresión de poca profesionalidad y desorden, lo que dificultará el seguimiento del formulario.
- Falta de información: Poca o nula retroalimentación frustrará al usuario al no saber si completó el formulario o si le faltó algún campo por rellenar.
- Ignorar a los usuarios móviles: Para que los usuarios móviles estén contentos nuestro formulario tiene que ser responsive, con campos claros y grandes para pulsarlos con mayor facilidad, usar ventanas emergentes en lugar de pequeños desplegables
- No probar los formularios: Es fundamental probar tu formulario en la mayor cantidad de situaciones posibles.
- Ignorar la accesibilidad: El formulario tiene que poder ser usado por cualquiera, para ello tener en cuenta en el diseño a personas con diferentes problemas de visión o garantizar que se pueda navegar solo con teclado garantizará que un mayor número de personas pueda rellenar el formulario sin complicaciones.
- No hay indicadores de progreso: En formularios largos la falta de indicadores de progreso como barras puede hacer que los usuarios se sientan perdidos.
- Captchas complicados: Usar captchas sencillos mantienen la seguridad mientras no ahuyentan a los usuarios.

10. Recomendaciones prácticas

Para cerrar el tema de formularios, imos ver algunas recomendaciones y buenas practicas. Entre paréntesis se ve el apartado donde se explica cada punto.

Antes de diseñar

- Justificar cada campo: si no sabemos para qué sirve un dato, no se pide (2).
- Decidir qué campos son obligatorios y dejar claro cuáles son opcionales (5).

Estructura

- Una sola columna, en orden lógico y con los grupos marcados con `<fieldset>` y `<legend>` (2).
- Mostrar solo los campos que aplican; en formularios largos, dividir en pasos con un indicador de progreso (2, 9).

Etiquetas y controles

- Etiquetas breves, descriptivas, visibles y asociadas a su campo con `for` e `id` (3).
- Botones de opción para elegir una sola opción entre pocas, casillas para elecciones independientes y desplegables solo con muchas opciones (4).
- El tipo de `<input>` adecuado a cada dato: `email`, `tel`, `date`… (4).

Validación y mensajes

- Validar cada campo cuando el usuario termina de rellenarlo, mostrar el error junto al campo y nunca solo con color (6).
- Mensajes concretos que expliquen cómo corregir el error (7).
- Validar siempre también en el servidor (4).

Usabilidad y accesibilidad

- Un recorrido lógico con la tecla Tab y un foco visible (8).
- Autorrelleno con `autocomplete`, salvo cuando perjudique al usuario (8).
- Sin botones de restablecer, y con campos y botones grandes en el móvil (8, 9).
- Probar el formulario con teclado, en el móvil y con usuarios reales (9).

Bibliografía / webgrafía

- Scottish Government Design System. *Form design*. designsystem.gov.scot/guidance/forms/form-design
- Scottish Government Design System. *Form structure*. designsystem.gov.scot/guidance/forms/form-structure
- Nielsen Norman Group. *Website Forms Usability: Top 10 Recommendations*. nngroup.com/articles/web-form-design
- Seckler, M. et al. (2014). *Designing usable web forms: empirical evaluation of web form improvement guidelines*. CHI 2014 (citado por Nielsen Norman Group).
- Nielsen Norman Group. *Checkboxes vs. Radio Buttons*. nngroup.com/articles/checkboxes-vs-radio-buttons
- MDN Web Docs. *HTML input types*. developer.mozilla.org
- Apple Computer (1984). *Inside Macintosh* (citado por Nielsen Norman Group).
- Nielsen Norman Group. *10 Design Guidelines for Reporting Errors in Forms*. nngroup.com/articles/errors-forms-design-guidelines
- web.dev (Google). *Help users avoid re-entering data in forms* (curso *Learn Forms*). web.dev/learn/forms/auto
- https://www.justinmind.com/es/blog/diseno-formulario/